

How to connect to the remote linux server?

ssh client

putty/ MobaXterm/ git bash-> windows user

ip address:

username: ubuntu

password: .pem file (devops.pem)

ssh -i devops.pem ubuntu@<ip>

Version:

Helps to track the changes we are making to a software over the period of time.

Significance of digits in version e.g. IOS 13.5.1

13-major (n number of features added, improve performance)

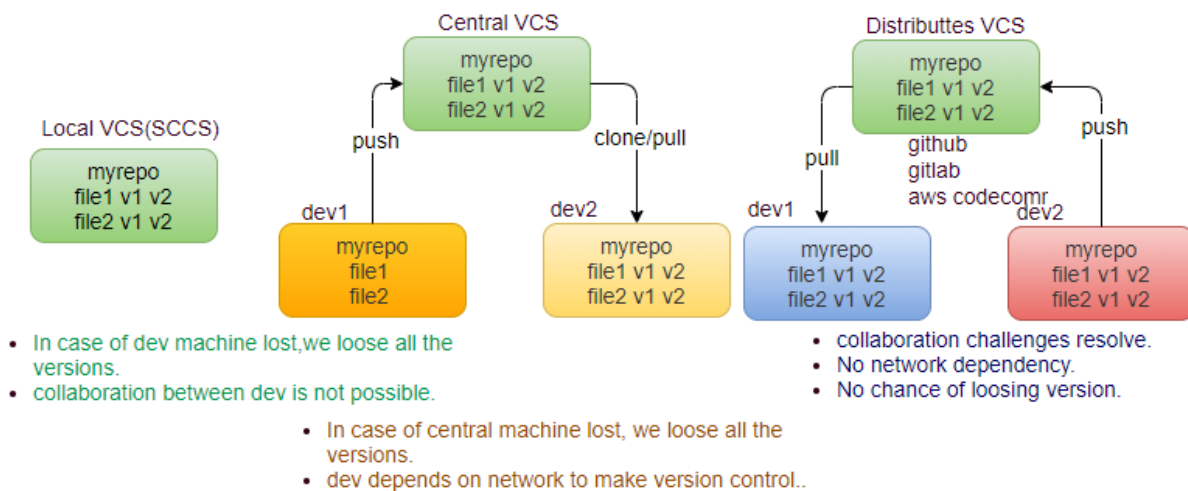
5- minor (bundle number of defects and simple use case)

1- HotFix (minor bug fixes)

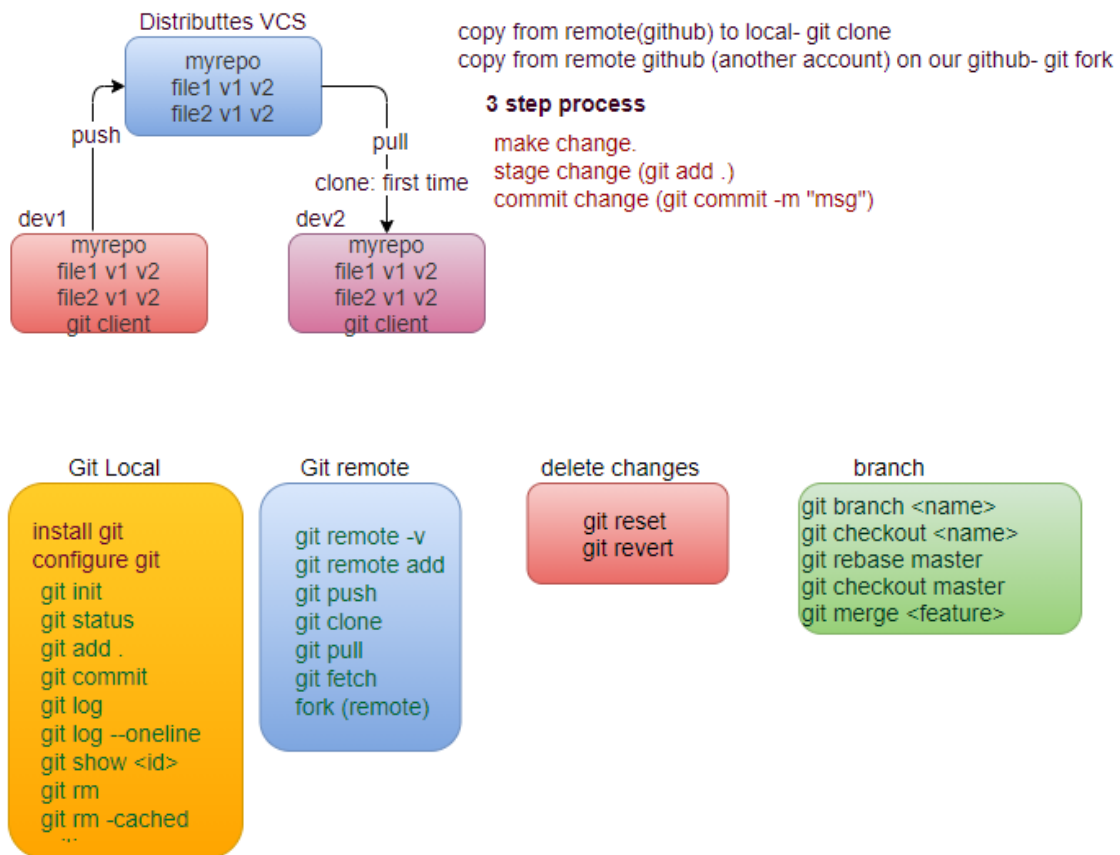
Version Control System (VCS)/Source Code Management (SCM) is a tool that helps to track the changes happening to file in your source code- **GIT**

Types of version control tools

- **local vcs- SCCS** Source Code Control System
- **centralized vcs-** (svn/ tfs/cvs)
- **distributed vcs-** (git/mercurry)



make change (CRUD?)



Google Cloud

Create VM instance (Ubuntu 18.04)

Step1: Install git client and configure

git config --global user.name neeraj // in tracking will show the details changes done by whom

git config --global user.email neerajvohra7419@gmail.com

git config --list

Step2: initialize local repository, any changes in this directory tracked by Git.

mkdir myproject

cd myproject

git init

#Hey! git track the changes (CRUD) in particular directory (creates local repo. git)

```

root@dev1:~# mkdir myproject
root@dev1:~# ls -l
total 8
drwxr-xr-x 16 root root 4096 Jan  1 17:05 labs
drwxr-xr-x  2 root root 4096 Jan  3 15:56 myproject
root@dev1:~# cd myproject/
root@dev1:~/myproject# pwd
/root/myproject
root@dev1:~/myproject# git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:     git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:     git branch -m <name>
Initialized empty Git repository in /root/myproject/.git/

```

make changes (create/update/delete any file)

Work area- any change (red color not tracked by git)

Staging area- git get access (green color), git start track changes, Identify the changes.

Commit area: Git Says I know all the changes, creates a version history

- local repository (creating a version history)

git status

every commit creates a new version id, HEAD represents the latest version id.

How to check commit area?

git log --help

git log

Shows all the commit id (**version tracking system**) HEAD refers to the recent commit id

git log --oneline #only commit id

git commit -am "message"

add and commit in one command -a (all can't specify specific file)

git log <commit Id> # who did the commit (version history)

What change did in particular commit id?

git show <commit Id>

How to avoid some files for tracking in my repository?

touch .gitignore #. <file> as a hidden file

nano .gitignore

neeraj.info // file

*.log // wild character

test/ //folder

cat .gitignore

How to see hidden files (.) in directory?

ls -al

All files inside .gitignore will be ignored (not tracked by git), .gitignore file specific to project but not to specific user.

Can we un-track the already committed file?

No, because git is already tracking it.

How can we refer each commit as a version number instead of commit id (alias name for long commit id?)

Tagging:

Every commit id refer to a new version, Tag refer to the release number.

git tag -a "v1.5" -m "version 1.5" // alias name for long commit id

git tag -a // took the recent commit id always

```
root@ip-172-31-31-147:~/myproject# git log --oneline
f4880d0 (HEAD -> master) fifth commit
53a0f4b gitignore
6cb35f8 third commit
150b078 second commit
a222d92 new file
root@ip-172-31-31-147:~/myproject#
root@ip-172-31-31-147:~/myproject# git tag -a "v1.5" -m "version 1.5"
root@ip-172-31-31-147:~/myproject#
root@ip-172-31-31-147:~/myproject# git log --oneline
f4880d0 (HEAD -> master, tag: v1.5) fifth commit
53a0f4b gitignore
6cb35f8 third commit
150b078 second commit
a222d92 new file
```

By default it takes a latest commit id to tag the version number.

How to tag specific commit id with version no?

git tag -a "v1.0" <commit id> -m "version 1.0" // alias

```
root@ip-172-31-31-147:~/myproject# git tag -a "v1.0" 6cb35f8 -m "version 1.0"
root@ip-172-31-31-147:~/myproject#
root@ip-172-31-31-147:~/myproject#
root@ip-172-31-31-147:~/myproject# git log --oneline
f4880d0 (HEAD -> master, tag: v1.5) fifth commit
53a0f4b gitignore
6cb35f8 (tag: v1.0) third commit
150b078 second commit
a222d92 new file
```

Now user can easily know that code till commit id 53a0f4b corresponds to version 1.0.

Can we automate the tagging?

No first we have to commit, we tag only commit id, not the file.

How to see differences between two versions?

git diff v1.5 v1.0

REMOTE REPOSITORY

step1: create a repository in a remote

`git remote add origin < remote repository path>` # where to upload the changes.

`git remote -v`

(local repository doesn't know where have to upload the changes)

`git push -u origin master` # -u upstream origin-> remote URL

As of now we have uploaded everything from our local system to local git repo.

Whatever committed will be push on to the remote repository.

```
root@dev1:~/myproject# git remote -v
root@dev1:~/myproject# git remote add origin https://github.com/lerndevops/myproject.git
root@dev1:~/myproject#
root@dev1:~/myproject# git remote -v
origin  https://github.com/lerndevops/myproject.git (fetch)
origin  https://github.com/lerndevops/myproject.git (push)
root@dev1:~/myproject# git branch -M main
root@dev1:~/myproject#
root@dev1:~/myproject# git push -u origin main
Username for 'https://github.com': lerndevop
```

`git branch -M main` # create a main branch from default master

`git push -u origin main` # u upstream, origin remote repository URL

UC: Consider that a new developer joined our team, he also has to install git client, and firstly he will make a local copy from the remote (entire project's copy).

`git clone` - first time operation, along with all commit id (versions).

`git log` // all the versions (commit id)

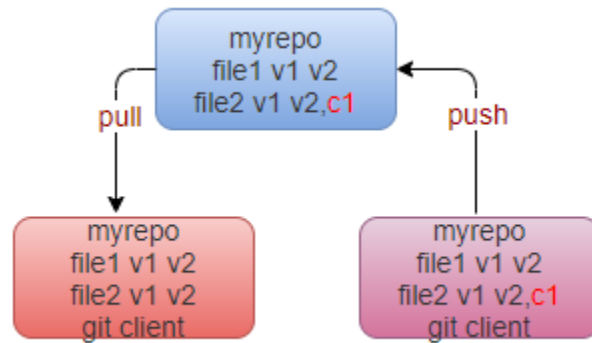
`git add dev2.java`

`git commit -m "dev2 changes"`

`git log` # header incremented in Dev2 Machine.

dev2 push his/her changes to Github, changes are pushed along with commit id.

UC: Now developer2 have done some changes and pushed to remote, developer1 wants to add only those changes in his local repository, not the whole repository?



git pull

We taking the changes from remote repository and adding it into local repository (also merging), also update the HEAD.

```
root@ip-172-31-31-147:~/myproject# git log --oneline
a4dd94b (HEAD -> master, origin/master, origin/HEAD) Create devNewFile.txt
f4880d0 fifth commit
53a0f4b gitignore
6cb35f8 third commit
150b078 second commit
a222d92 new file
```

UC: Dev1 did some changes in 1.java file and pushed it on the remote.

Dev2 don't want to take latest changes by dev1, but wants to know is there any changes in the remote which are not available in his local?

git fetch

doesn't update the local repository, just tell that there is some change in remote if any.

```
root@ip-172-31-31-147:~/myproject# git fetch
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
Unpacking objects: 100% (3/3), 667 bytes | 667.00 KiB/s, done.
From https://github.com/lerndevops/myproject
 f4880d0..a4dd94b master -> origin/master
```

fetch just tell there are change in your local HEAD (master) and remote HEAD (origin/master), git fetch doesn't get the changes and add into your local **just tell** that there is something new in remote repository.

UC2: dev1 has pushed some change on the remote, dev2 trying to push his/her changes (without taking the latest pull from remote)


```

Untracked files:
  (use "git add <file>..." to include in what will be committed)
  day2.txt

nothing added to commit but untracked files present (use "git add" to track)
root@instance:~/myproject# vi 1.java
root@instance:~/myproject# git add .
root@instance:~/myproject# git status
On branch master
Your branch is up to date with 'origin/master'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    modified:   1.java
    new file:   day2.txt

root@instance:~/myproject# git commit -m "new"
[master e6360a6] new
 2 files changed, 4 insertions(+), 1 deletion(-)
 create mode 100644 day2.txt
root@instance:~/myproject# git push
Username for 'https://github.com': lerndevops
Password for 'https://github.com':
To https://github.com/lerndevops/myproject.git
 ! [rejected]        master -> master (fetch first)
error: failed to push some refs to 'https://github.com/lerndevops/myproject.git'
hint: Updates were rejected because the remote contains work that you do
hint: not have locally. This is usually caused by another repository pushi
hint: to the same ref. You may want to first integrate the remote changes
hint: (e.g., 'git pull ...') before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
root@dev2:~/myproject#

```

UC3: Both dev did changes in same file, dev1 has pushed the changes in remote. Now dev2, first pull the latest code (else conflict will be happen during push by dev2 because changed file already pushed by dev1)

```

-rw-r--r-- 1 root root 0 Nov 11 02:02 day2.txt
-rw-r--r-- 1 root root 0 Nov 11 02:07 dev2-nde2.yml
-rw-r--r-- 1 root root 0 Nov 11 01:52 devv2cchan.txt
-rw-r--r-- 1 root root 0 Nov 10 03:19 hello.java
root@instance:~/myproject# vi 1.java
root@instance:~/myproject# git status
On branch master
Your branch is up to date with 'origin/master'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be com
  (use "git restore <file>..." to discard changes in
    modified:   1.java

no changes added to commit (use "git add" and/or "git
root@instance:~/myproject# git add .
root@instance:~/myproject# git commit -m "dev1"
[master 0ab2866] dev1
 1 file changed, 4 insertions(+)
root@instance:~/myproject# git push -u origin master
Username for 'https://github.com': lerndevops
Password for 'https://github.com':
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 2 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 298 bytes | 298.00 KiB/s
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with
To https://github.com/lerndevops/myproject.git
 cd0b521..0ab2866 master -> master
Branch 'master' set up to track remote branch 'master'
root@dev2:~/myproject#
-rw-r--r-- 1 root root 0 Nov 11 01:47 4.xml
-rw-r--r-- 1 root root 0 Nov 11 02:06 day2.txt
-rw-r--r-- 1 root root 0 Nov 11 02:04 dev2-nde2.yml
-rw-r--r-- 1 root root 0 Nov 11 01:48 devv2cchan.txt
root@dev2:~/myproject# cat 1.java
hi there

hello how are you !!
root@dev2:~/myproject# vi 1.java
root@dev2:~/myproject# git add .
root@dev2:~/myproject# git commit -m "hello"
[master 73c9e7a] hello
 1 file changed, 3 insertions(+)
root@dev2:~/myproject# git pull
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 1), reused 3 (delta 1), pack-reused 0
Unpacking objects: 100% (3/3), done.
From https://github.com/lerndevops/myproject
 cd0b521..0ab2866 master -> origin/master
Auto-merging 1.java
CONFLICT (content): Merge conflict in 1.java
Automatic merge failed; fix conflicts and then commit the result.
root@dev2:~/myproject#

```

```
hi there  
e  
hello how are you !!  
  
<<<<<<< < HEAD  
new code from dev2  
=====
```

new code change dev1

```
>>>>>>> 0ab28660f9b9f12a9cd3ed9e5a90e779fe386767
```

```
root@instance:~# cd myproject/
root@instance:~/myproject# ls -l
total 4
-rw-r--r-- 1 root root 56 Nov 11 02:09 1.java
-rw-r--r-- 1 root root 0 Nov 10 03:03 2.java
-rw-r--r-- 1 root root 0 Nov 10 03:07 3.yml
-rw-r--r-- 1 root root 0 Nov 10 03:07 4.xml
-rw-r--r-- 1 root root 0 Nov 11 02:02 day2.txt
-rw-r--r-- 1 root root 0 Nov 11 02:07 dev2-nde2.yml
-rw-r--r-- 1 root root 0 Nov 11 01:52 devv2cchan.txt
-rw-r--r-- 1 root root 0 Nov 10 03:19 hello.java
root@instance:~/myproject# cat 1.java
hi there

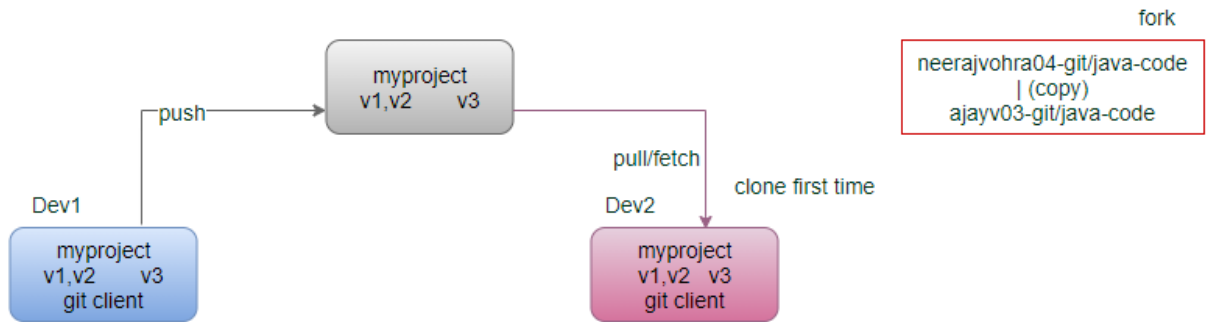
hello how are you !!

new code change dev1
root@instance:~/myproject#
```

To contributes towards the open source code.

remote party will receive a pull request, after receiving the pull request remote party can merge/deny the code in its own repository. **clone is for local (owner), fork is for remote account (don't have write access).**

Fork, it will make a copy from remote account to our account. Fork is operation to make copy of public resource in your account (clone).



step1: Login into your account.

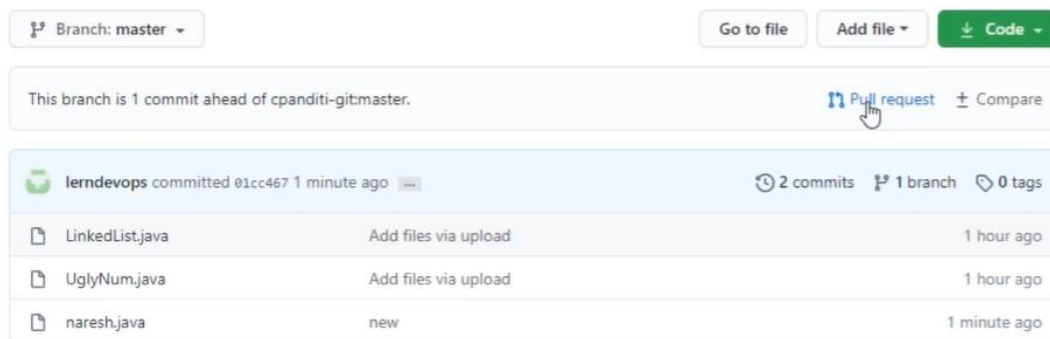
step2: Go to open-source link (third party publically access repository)

step3: Click on fork (will copy the remote repo in your git hub account)

step4: contribute to open source by cloning and adding some new utilities, and commit the same in your local repo.

step5: push the changes in your git hub account.

step6: raise a pull request to third party (owning the open-source code), can't directly upload the change only can request from owner of the repository.



It's the way to contributing to open source.

step7: A new pull request available to the third-party account which either can be accepted by merging or rejected by the third party.

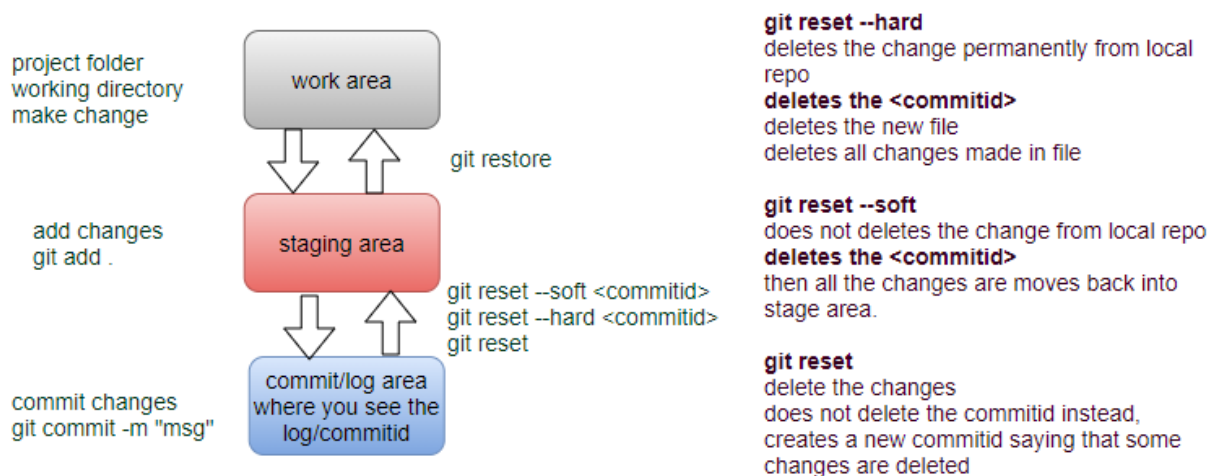
Rolling back the changes.

`touch abc.txt`

`git add .` // work area -> stage area

`git commit -m "rollback"` // stage area -> commit area

`git status`



UC: revert the change: we want to delete the commit id and revert back to staging area from commit area. (Before pushing to remote)

```
root@ip-172-31-31-147:~/myproject# git log --oneline
a4dd94b (HEAD -> master, origin/master, origin/HEAD) Create dev1newfile.txt
f4880d0 fifth commit
53a0f4b gitignore
6cb35f8 third commit
150b078 second commit
a222d92 new file
```

We want to revert back the commit id a4dd94b

touch new

git add .

git commit -m "add new file"

```
root@ip-172-31-31-147:~/myproject# git log --oneline
0b4ebdd (HEAD -> master) new
a4dd94b (origin/master, origin/HEAD) Create dev1newfile.txt
f4880d0 fifth commit
53a0f4b gitignore
6cb35f8 third commit
150b078 second commit
a222d92 new file
root@ip-172-31-31-147:~/myproject#
```

git reset --soft <commit id> #commit id where we want to reach.

git reset --soft a4dd94b # commit area to staging area

git status

UC: revert the change: we want to revert back to work area from staging area

git restore --staged <file name>

git restore --staged abc.txt

git status

UC: Now delete from work area

rm abc.txt

ls-l

UC: how can we delete from all area in one shot?

touch abc.html new.yml

git status

git add .

git status

git commit -m "hard reset"

git status

git log --oneline

git reset --hard <commit Id>

Will be delete from everywhere including commit id (delete from the repository)

What is git stash command?

UC: fileA is in staging area we haven't committed yet, meanwhile we have received a blocker defect, which we have to deliver on priority. In this case we have to commit the defect code on priority, and also to ensure that fileA code neither to be committed, nor to be restore back to working area.

```
root@instance:~/myproject#  
root@instance:~/myproject# git status  
On branch defect  
Your branch is up to date with 'origin/defect'.  
  
Changes to be committed:  
  (use "git restore --staged <file>..." to unstage)  
    new file:   dfec.java  
  
Changes not staged for commit:
```

git stash: will move all the stage files in special area called stash, will not be candidate for commit.

```
root@instance:~/myproject#  
root@instance:~/myproject# git stash  
Saved working directory and index state WIP on defect: flf71fc defect  
root@instance:~/myproject#  
root@instance:~/myproject# git status  
On branch defect  
Your branch is up to date with 'origin/defect'.  
  
nothing to commit, working tree clean
```

UC: If we want only newfile move to stash area

git stash newfile

git commit -m "production defect code"

Now commit command will only commit the defect.java

git log --oneline

git push

UC: How newfile will be back from stash area in future, after delivering the defect code?

git stash list

```
root@instance:~/myproject# git stash list
stash@{0}: WIP on defect: flf71fc defect
root@instance:~/myproject#
root@instance:~/myproject#
root@instance:~/myproject# git stash show stash@{0}
dev2-nde2.yml | 1 +
dfec.java      | 0
2 files changed, 1 insertion(+)
```

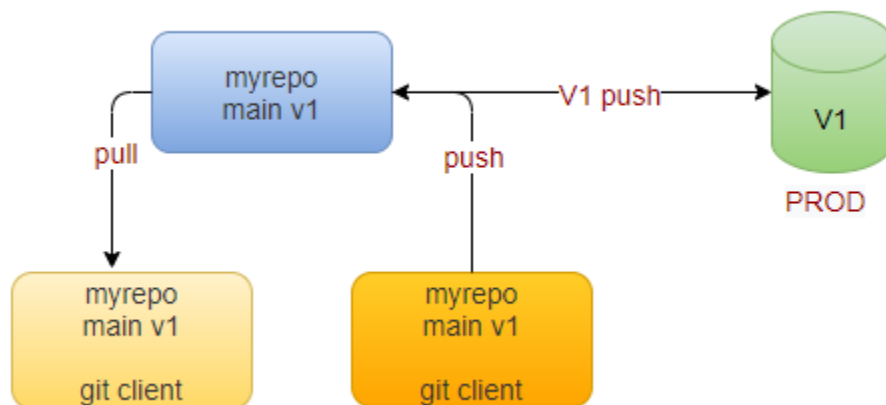
git stash show <stash@{0}> # f.txt available

git stash apply <stash@{0}> # back again into staging area

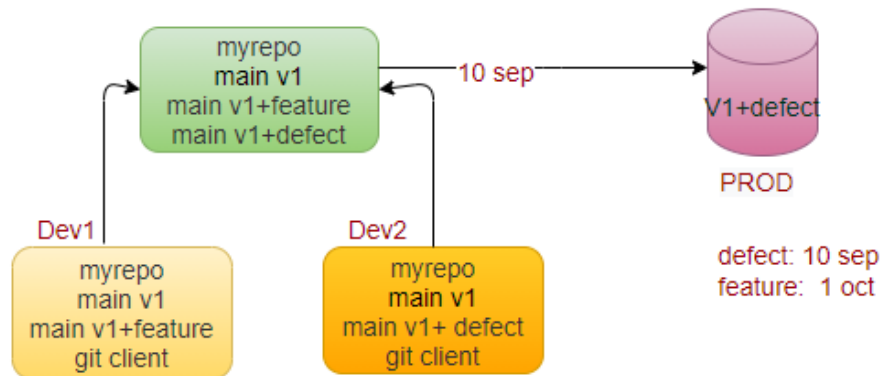
git status # feature file available again in staging area

Branching: Why do we need branching (by developers)?

UC: Team have pushed our code in GitHub, v1 is successfully deployed on the production



Once code is delivered, team has started work on new feature feaxy, meanwhile client reports a defect on v1 code, and now team has to work on both new feature as well as defect.



Problem: in git hub main branch we have code of defect as well as of feature, how will move the only code written for defect (v1+ defect code) on the production on 10th Sep?

Solution: Branching, makes your release smooth. Instead of working on the main branch (original code), each developer work on their own copy of the main branch, it is always a copy of the original (main branch).

The developer 1 has featxy branch (copy of main branch) and developer 2 has defect branch (copy of main branch), and both start working on their separate branch.

The developer 1 push his code on remote and developer 2 push his code on the remote. Now in master we have three copies (versions of the code).Three copies of the code.

Our objective to deploy only 1 version (defect v1 + d1)

UC: First create a branch in Remote by admin and team member clone the specific branch in their local.

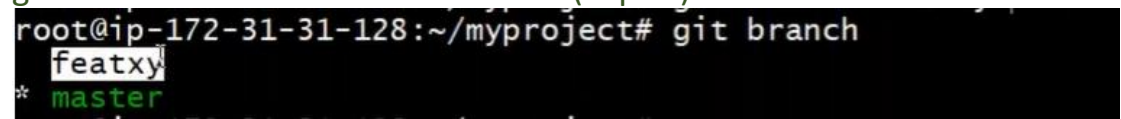
Clone of specific branch

`git clone -b featxy <URL>`

Strategy: merge the defect copy version with master copy version and deploy, always deploy master version.

```
mkdir myproject
cd myproject
git init
# By default, whenever we initialize the git will get the master branch is the
# default branch
# can be change to main branch
git branch -M main          # will change the master branch to main branch
git status                  # on branch master/main
touch 1.java 2.java
git add .
git commit -m "m1"          # m1 master branch
touch 3.java 4.java
git add .
git commit -m "m2"          # m2 master branch
git branch                  # existing branches with current branch*
git log --oneline           # 2 commits corresponding to v1
git push -u origin master/main
```

```
git branch feature # will create a copy of the branch from where it executed
git branch          # have two branches (copies) master and feature
```



```
root@ip-172-31-31-128:~/myproject# git branch
  featxy
* master
```

```
git checkout feature          # switch to feature branch
git branch
git ls -l                     # same as master
git log --oneline             # same as master branch(copy)
```



```

root@ip-172-31-31-128:~/myproject# git branch
* featxy
* master
root@ip-172-31-31-128:~/myproject# ls -l
total 0
-rw-r--r-- 1 root root 0 Jul 11 15:24 1.java
-rw-r--r-- 1 root root 0 Jul 11 15:24 2.java
-rw-r--r-- 1 root root 0 Jul 11 15:24 3.java
-rw-r--r-- 1 root root 0 Jul 11 15:24 4.java
root@ip-172-31-31-128:~/myproject# git log --oneline
2795c7e (HEAD -> master, origin/master, featxy) m2
1e2fb83 m1
root@ip-172-31-31-128:~/myproject#
root@ip-172-31-31-128:~/myproject# git checkout featxy
Switched to branch 'featxy'
root@ip-172-31-31-128:~/myproject# git branch
* featxy
  master
root@ip-172-31-31-128:~/myproject#
root@ip-172-31-31-128:~/myproject# ls -l
total 0
-rw-r--r-- 1 root root 0 Jul 11 15:24 1.java
-rw-r--r-- 1 root root 0 Jul 11 15:24 2.java
-rw-r--r-- 1 root root 0 Jul 11 15:24 3.java
-rw-r--r-- 1 root root 0 Jul 11 15:24 4.java
root@ip-172-31-31-128:~/myproject# git log --oneline
2795c7e (HEAD -> featxy, origin/master, master) m2
1e2fb83 m1

```

any changes done in any branch will not be reflected in any other branch.

touch f1.java

git add .

git commit -m "f1 in feature branch"

touch f2.java

git add .

git commit -m "f2 in feature branch"

will not reflect in master branch (locally or remotely)

git log --oneline // HEAD is moved to f2

```

root@ip-172-31-31-128:~/myproject# git log --oneline
c363819 (HEAD -> featxy) f2
342c271 f1
2795c7e (origin/master, master) m2
1e2fb83 m1
root@ip-172-31-31-128:~/myproject# ls -l
total 0
-rw-r--r-- 1 root root 0 Jul 11 15:24 1.java
-rw-r--r-- 1 root root 0 Jul 11 15:24 2.java
-rw-r--r-- 1 root root 0 Jul 11 15:24 3.java
-rw-r--r-- 1 root root 0 Jul 11 15:24 4.java
-rw-r--r-- 1 root root 0 Jul 11 15:30 featx.java
-rw-r--r-- 1 root root 0 Jul 11 15:31 featy.java

```

git checkout master #switching to master

ls -l # same as previous no change in master branch

git log --oneline #same as previous no change in master branch

We have two distinct copy (version) available on my local.

```

root@ip-172-31-31-128:~/myproject# git checkout master
Switched to branch 'master'
Your branch is up to date with 'origin/master'.
root@ip-172-31-31-128:~/myproject#
root@ip-172-31-31-128:~/myproject# ls -l
total 0
-rw-r--r-- 1 root root 0 Jul 11 15:24 1.java
-rw-r--r-- 1 root root 0 Jul 11 15:24 2.java
-rw-r--r-- 1 root root 0 Jul 11 15:24 3.java
-rw-r--r-- 1 root root 0 Jul 11 15:24 4.java

```

git push -u origin feature #feature branch will be uploaded

Now in remote we have 2 copies, we can directly release this feature branch in production if we want.

UC: we want to deploy master/main branch, in production always deploy the master/main branch, how to push the changes in to master branch?

We can do the same either in remote or on local, generally **merging** happen at **remote location**.

Consider the scenario on the remote, **in remote** have option of pull request. In local there is no concept of pull /merge request.

The screenshot shows a GitHub repository interface for a branch named 'featx'. At the top, it indicates '2 branches' and '0 tags'. Below this, a message states 'This branch is 2 commits ahead of master.' To the right of this message are links for 'Pull request' and 'Compare'. The main section of the interface displays a list of files and their commit history. The files listed are 1.java, 2.java, 3.java, 4.java, featx.java, and featy.java. Each file is associated with a commit hash (m1, m2, f1, f2) and a timestamp (15 minutes ago or 9 minutes ago). The repository name 'lerndevops f2' is visible at the top left of the file list, along with the commit hash 'c363819' and the time '9 minutes ago'.

File	Commit	Time
1.java	m1	15 minutes ago
2.java	m1	15 minutes ago
3.java	m2	15 minutes ago
4.java	m2	15 minutes ago
featx.java	f1	9 minutes ago
featy.java	f2	9 minutes ago

You don't have access to the master/main branch. In that case you need request to the member who have access to the master/main branch, request done through the pull request (request to the owner of the master/main branch, saying that I have done some changes, you can review it and can add into the master). When you create a pull request it goes into the pull request page of the owners of the master/main branch.

Owner will review the changes once he submit the review request, will get the option of merge the request on clicking the merge pull request get option of confirm merge request as owner confirms the merge request feature code merge with master branch. Owner also can return back the request with some suggestion. Once you send a pull request your branch becomes locked and will be unlocked once your request will be accepted or returned back. You can also **close pull request**.

Once you finalize your changes only then create a pull request.

How to merge in local? No approval concept

scenario1:

master: m1 m2

feature: m1 m2 f1 f2

git checkout master # switch to master branch

git merge feature

master: m1 m2 f1 f2

scenario2

master: m1 m2

feature: m1 m2 f1 f2 will be release on Oct-1st by Dev1

defect m1 m2 d1 d2 will be released on 10th Sep by Dev2

On 10th Sep

master: m1 m2 d1 d2

feature: m1 m2 f1 f2 will be release on Oct-1st by Dev1

On 1st Oct

git checkout master

git merge feature

What will be the master, because there is something extra d1, d2 in master which is not available in feature branch, there are lots of options, but for clean history option1 is correct?

master: m1 m2 d1 d2 f1 f2

m1 m2 f1 f2 d1 d2

m1 m2 d1 d2 f1 f2 e1

m1 m2 f1 f2 d1 d2 e1

There is issue in this order if someone see the history

e1 commit id is useless because it doesn't refer to any change in source code, git automatically generates an unnecessary commit id against any merge request, order of history is misleading.

Best Practice

When you create a feature branch from master branch (m1 m2), and when you are merging back your feature branch (m1 m2 f1 f2) with the master branch (**from where you have created a branch**), you have to ensure always merge with the latest version of the master branch (or because as per our above scenario, it may be possible that after creating a feature branch from the master branch some other developer has already pushed the changes on the master branch (defect branch), means the currently master version (m1,m2,d1,d2) is not same as at the time of you have created your branch (m1,m2).

Means before merging first pull the latest code and after that merge your code in that code.

base of the feature branch is the HEAD of the master from where branch is created, by default merge always merged the code on base of the feature branch's HEAD.

Before the merging ensure that base of my feature branch is always the latest of the master branch.

On 10th Sep

master: m1 m2 d1 d2

feature: m1 m2 f1 f2 will be release on Oct-1st by Dev1

On 1st Oct

git checkout master

git merge feature **m1 m2 f1 f2 d1 d2 e1**

There is issue in this order if someone see the history

Solution: Rebase

On 1st Oct

master: m1 m2 d1 d2

feature: m1 m2 f1 f2

```

root@ip-172-31-31-128:~/myproject# git log --oneline
a51648b (HEAD -> master) d1
2795c7e (origin/master) m2
1e2fb83 m1
root@ip-172-31-31-128:~/myproject# git checkout featxy
Switched to branch 'featxy'
Your branch is up to date with 'origin/featxy'.
root@ip-172-31-31-128:~/myproject# git log --oneline
c363819 (HEAD -> featxy, origin/featxy) f2
342c271 f1
2795c7e (origin/master) m2
1e2fb83 m1

```

git checkout **feature** # switch to feature branch

git log --oneline

m1 m2 f1 f2

Before your current branch (feature) merge back to the same branch (master), first take a current latest from same branch (master), and rebase your current branch (feature) with master.

git rebase master # you have cleaner history local->local in feature branch

git log --oneline

m1 m2 d1 d2 f1 f2

```

root@ip-172-31-31-128:~/myproject# git rebase master
Successfully rebased and updated refs/heads/featxy.
root@ip-172-31-31-128:~/myproject# git log --oneline
de29595 (HEAD -> featxy) f2
130d7f3 f1
a51648b (master) d1
2795c7e (origin/master) m2
1e2fb83 m1

```

git checkout **master** # switch to master branch

git merge **feature**

git log --oneline

m1 m2 d1 d2 f1 f2

rebase ensures the clean history, hence before merging any feature branch in master first rebase the feature branch with master will ensure that will maintain the clean history.

How to delete the branch

git checkout master

git branch

git branch -D feature

If will try to delete master branch from within master branch will not deleted

We can also delete the feature branch from the remote.

Merge Conflicts

git branch # master

git branch defect # create a new branch defect

git branch # master, defect

master/main	Defect
1.java	1.java
2.java	2.java
3.defect.java	3.defect.java
4.featx.java	4.featx.java
modify defect.java	modify defect.java
git add .	git add .
git commit -m "conflict"	git commit -m "conflict"

UC: You are working on defect.java file in defect branch and at same time some other developer has also updated the same defect.java file in master branch. Or it may be scenario that both are working on different features but working in the same file. It must be the same file.

git checkout defect

nano defect.java

git add .

git commit -m "change in defect branch-conflict"

git checkout master

nano defect.java

git add .

git commit -m "change in master branch-conflict"

We have modified the same file in both branches. Merging is always getting the changes from one branch to other branch.

git checkout master

git merge defect #automatic merge failed, except the file have conflict remaining will merged

How to solve?

step1: Open the file have merge conflict (defect.java)

nano defect.java

```
<<<<<< HEAD
hi there !! maaste change !!
=====
hi there!! change at defect branch !!
>>>>>> defect
```

git status
git add .
git commit -m "resolve merge conflict"

Git workflow

master == main

Git Workflows

Centralized workflow (small project)

remote master branch --> local master branch -- make changes in local -- push back to remote master -- release remote master into PROD

Feat workflow (medium size projects)

remote master -- create feat/defect branch -- dev works on feat/defect -- push back to feat/defect -- release feat/defect branch -- merge to the remote master

remote master -- create feat/defect branch -- dev works on feat/defect -- push back to feat/defect -- merge the changes to remote master -- then release remote master into PROD

git flow workflow

remote master -- remote release branch -- remote feat/defect -- dev work on feat/defect -- push back to feat/defect -- merge to release branch -- release the release branch into prod -- merge release branch into master branch

